Software Maintenance — Models, Categories, and Descriptions

Summary Table: Models × Categories × Usage

Model	Description	Best for Categories	When to Use	Why to Use	Typical Activities	Examples
Maintenance Process Model 1 (Small Rework)	Lightweight, direct edits to existing code; update docs after change; integrate and test	Corrective (small), Adaptive (minor), Perfective (minor), Preventive (small refactors)	Changes are localized, low-risk, and do not affect architecture; system and docs are understandable; quick turnaround needed	Fast, low overhead; leverages existing code knowledge; efficient for incremental changes	1) Gather changed requirements 2) Analyze change 3) Devise code-change strategy 4) Modify code 5) Update documents 6) Integrate and test	Bug fix in one module; minor performance tuning; small UI text/layout change; config tweak for a new endpoint

Maintenance Process Model 2 (Significant Rework / Reengineering)	Reverse engineering to recover specs/design → apply change requests to derive new requirements → forward engineering to redesign/implement; reuse artifacts	Corrective (systemic), Adaptive (major), Perfective (architectural), Preventive (large-scale refactor)	Broad or architectural changes; inadequate/obsolete documentation; legacy/technical debt; platform/OS/compiler/hardware migration; major feature redesign	Reduces risk by rebuilding understanding first; enables modernization and improved quality; better long-term maintainability	Reverse: Code → Module specs → Design → Original requirements → Apply change requests → New requirements. Forward: New requirements → Design → Module specs → Code	Migrating to new OS/hardware/cloud; decomposing monolith; core domain redesign; pervasive reliability/performance fixes

---

Maintenance Categories — What, When, Why, Examples

Category	What it is	When it applies	Why it's needed	Typical examples
Corrective	Fixing faults discovered in operation	Production defects, incorrect outputs, crashes	Restore correct behavior and service quality	Patch a null pointer bug; fix wrong tax

	(emergency or scheduled repairs)			calc; resolve a crash path
Adaptive	Modifying software to new environments (platforms, OS, compilers, hardware) or external requirements	Platform/stack upgrade, API/standard changes, regulatory/environment shifts	Keep system operational amid environmental changes	Port to new OS; migrate DB engine; update to new payment API
Perfective	Improving performance, structure, reliability, or adding new features	User feedback, performance/UX goals, maintainability improvements	Increase value, usability, and efficiency	Add reporting feature; optimize query plan; simplify module interfaces
Preventive	Addressing latent issues to avoid future failures	Early signs of complexity/fragility;	Reduce future maintenance cost and risk	Large refactor of shared library; add missing tests;

	(reduce technical debt)	recurring minor issues; aging code hotspots		improve error handling

---

How Models Map to Categories (Quick Guide)

- Corrective:
  - Small, localized bug fixes → Model 1.
  - Cross-cutting or design-rooted defects → Model 2.
- Adaptive:
  - Minor adjustments (config, small library bump) → Model 1.
  - Major platform/OS/compiler/hardware migration → Model 2.
- Perfective:
  - Minor performance/UX improvements → Model 1.
  - Architectural optimization or modularization → Model 2.
- Preventive:
  - Small refactors, code clean-ups → Model 1.
  - Large-scale refactoring or architecture hardening → Model 2.

Model Descriptions

Maintenance Process Model 1 (Small Rework)

- Description:
  - A pragmatic, low-overhead loop: collect changed requirements, analyze the change and impact, choose code-change strategies, apply changes in the existing codebase, update documentation, then integrate and test.
- Use when:
  - Scope is limited and well understood; architecture remains intact; risk is low; a quick fix or incremental enhancement is needed; code and docs are understandable (ideally with some original team insight).
- Benefits:
  - Faster, cheaper for small scope; minimizes process overhead while ensuring verification and documentation alignment.
- Steps:
  - Gather changed requirements
  - Analyze changes and impact

        - Devise code-change strategies
        - Apply changes to old code
        - Update documents
        - Integrate and test
- Examples:
    - Fix an algorithm edge case in one function; improve a single slow query; add a small UI validation; rotate API keys via configuration.

Maintenance Process Model 2 (Significant Rework / Reengineering)

- Description:
    - A two-phase approach for high-impact change. Reverse engineering recovers knowledge from code (module specs → design → original requirements). Apply change requests to produce new requirements. Forward engineering redesigns and reimplements, reusing reverse-engineered artifacts.
- Use when:
    - Changes are architectural or widespread; documentation is lacking or outdated; legacy code carries heavy technical debt; platform migrations or major features demand redesign.
- Benefits:
    - Reduces risk via structured understanding; enables modernization; improves maintainability and quality attributes (performance, reliability, security).
- Steps:
    - Reverse engineering: Code → Module specifications → Design → Original requirements → Apply change requests → New requirements.
    - Forward engineering: New requirements → Design → Module specifications → Code.
- Examples:
    - Move to a new OS/CPU or cloud provider; split a monolith into services; rework the core data model affecting many subsystems; address systemic latency or fault tolerance.

Choosing the Right Path — Decision Checklist

- Pick Model 1 if:
    - Change is small/contained; architecture stays the same; low risk; documentation/code are clear; speed matters.
- Pick Model 2 if:
    - Change is broad/architectural; documentation is weak/outdated; platform migration or major refactor/feature; substantial technical debt.